AWS · SERVERLESS · CASE STUDY

SFPS V1

I built a serverless file processing pipeline on AWS

S3 to Lambda to DynamoDB, with version-pinned reads, conditional writes, and a failure model that separates bad input from real faults.

135
automated tests passed

25
DynamoDB records from the deployed run

1
verified SQS failure envelope

Amazon S3 · AWS Lambda · DynamoDB · SQS · CloudWatch · IAM · Python 3.13

Sumanth R Malagar · 18 October 2026

SWIPE →

Uploading a file is easy. Processing it safely is not.

The pipeline had to answer five questions before it could be called complete.

- How do I get one deterministic record per S3 object version?
- What happens when S3 delivers the same event twice?
- How do I fail a bad image without retrying it forever?
- How does a real infrastructure fault retry and stay visible?
- How do I separate verified claims from documented configuration?

Five AWS services, one processing path

Uploader
AWS CLI or console

↓ upload to incoming/

Amazon S3
private · versioned · SSE-S3

↓ ObjectCreated:* event

AWS Lambda
sfps-processor · Python 3.13 · arm64

↓ conditional write

Amazon DynamoDB
one record per object version

from Lambda ↓

CloudWatch Logs
JSON lines · 30-day retention

Amazon SQS
sfps-failed-events

What happens after the upload

1. Claim a DynamoDB record with a conditional PutItem.
2. Read the size with a version-pinned HeadObject.
3. Reject zero-byte and oversized objects, then check the extension allowlist.
4. Fetch the object once with a version-pinned GetObject.
5. Validate the content, extract metadata, compute SHA-256.
6. Write the terminal state: COMPLETED or FAILED.

> Infrastructure exception → sanitized logs → re-raise → async retries → SQS on-failure destination. Permanent validation failures stop at the FAILED record.

IDEMPOTENCY

One record per S3 object version

```
processing_id = SHA-256({ bucket, object_key, version_id })
```

- Claim: PutItem conditioned on attribute_not_exists(processing_id).
- Terminal: UpdateItem guarded so a COMPLETED record can never be overwritten.
- Only COMPLETED is skipped. PROCESSING and FAILED records are re-attempted.
- ConsistentRead after a claim conflict stops stale reprocessing.

VERSION PINNING

Every read carries the event's version ID

- HeadObject and GetObject both receive the version_id from the S3 event.
- The bytes validated, hashed, and measured are the bytes the record identifies.
- A missing version ID is a malformed event, not a cue to read the latest version.

> The deployed table shows two sample.png versions: one COMPLETED, one FAILED on the 16,000,000-pixel budget. Same key, two independent records.

FAILURE MODEL

Permanent failures don’t retry. Real faults do.

Validation failure

Unsupported extension · corrupt image · invalid UTF-8 · zero bytes · oversized object

→ FAILED record.
→ No retry. No SQS.

Infrastructure failure

AWS service error · unexpected exception · invalid configuration

→ Sanitized logs.
→ Guarded FAILED attempt.
→ Re-raise, retry, then SQS.

The queue stays meaningful: a message means an operational fault, not an ordinary rejected upload.

What the deployed run actually shows

135

tests passed in 48.66 s · Ruff clean · no broken requirements

25

DynamoDB records: 10 COMPLETED, 15 FAILED

3

approximateInvokeCount in the captured SQS envelope (RetriesExhausted)

89 / 150.13 / 117

one PNG invocation: ms app · ms Lambda · MB peak, 512 MB configured

25 records are not 25 test scenarios: the set includes two versions of the same image and seven zero-byte folder markers.

The gaps are part of the result

- No production-grade security or fault-tolerance claim.
- Bucket policy, IAM attachment, and log retention are documented, not all visually verified.
- The 25 MiB memory and timeout boundary was not measured on the deployed function.
- No billing data, so cost is described qualitatively.

> The evidence inventory lists every capture I am missing and every claim I could not prove.

47 pages. 24 figures. 25 tables. One evidence trail.

Architecture, processing logic, packaging, security, the full test matrix, the captured SQS envelope, and the gaps I did not hide.

FULL PDF + HTML CASE STUDY

Link in the first comment

Includes the evidence inventory and both original architecture diagrams.